DATABASE STRUCTURE DOCUMENT

Mini Coffee POS & Inventory System

1. Database Overview

The database is designed for the Mini Coffee POS & Inventory System, a web-based POS and inventory management system for a small coffee shop. The main testing focus is the integration flow:

Product -> Recipe -> Inventory -> POS Order -> Stock Deduction -> Order History -> Reports

The database is hosted on Supabase PostgreSQL cloud. The system does not use local database storage. The backend connects to Supabase PostgreSQL through a database connection string and handles all business logic such as authentication, role checking, order validation, stock checking, stock deduction, and report calculation.

The database is intentionally kept simple because the project is for SWT301 software testing. It only covers the agreed project scope and does not include payment gateway, promotion, membership, delivery, AI prediction, or multi-branch features.

---

2. Main Database Objects

The database contains the following main objects:

No.	Object Name	Type	Purpose
1	app_users	Table	Store Admin and Staff accounts for login and authorization
2	products	Table	Store sellable coffee/drink products
3	ingredients	Table	Store ingredients, units, current stock, and low-stock threshold
4	recipes	Table	Store product recipe headers
5	recipe_items	Table	Store ingredients and required quantity for each recipe
6	orders	Table	Store successful POS orders
7	order_items	Table	Store product items inside each order
8	stock_transactions	Table	Store stock import, adjustment, and order deduction history
9	v_pos_available_products	View	Show products available for POS selling

No.	Object Name	Type	Purpose
10	v_low_stock_ingredients	View	Show ingredients below or equal to low-stock threshold
11	v_daily_revenue	View	Show daily revenue and order count
12	v_best_selling_products	View	Show best-selling product ranking
13	v_order_history	View	Show order history summary

---

3. Enum Types

3.1 user_role

Used in `app_users.role`.

Value	Meaning
ADMIN	System administrator / coffee shop manager
STAFF	Counter staff who creates POS orders

3.2 user_status

Used in `app_users.status`.

Value	Meaning
ACTIVE	User can login and use the system
INACTIVE	User cannot login

3.3 product_status

Used in `products.status`.

Value	Meaning
ACTIVE	Product is active and can be sold if it has a recipe
INACTIVE	Product is inactive and must not be sold on POS

3.4 ingredient_unit

Used in `ingredients.unit`.

Value	Meaning
GRAM	Ingredient measured by gram
ML	Ingredient measured by milliliter
PIECE	Ingredient measured by count/piece

3.5 stock_transaction_type

Used in `stock_transactions.type`.

Value	Meaning
IMPORT	Stock imported by Admin
ORDER_DEDUCT	Stock deducted after successful POS order
ADJUST	Manual stock adjustment

3.6 order_status

Used in `orders.status`.

Value	Meaning
SUCCESS	Successful order
CANCELLED	Cancelled order, optional for future use
REFUNDED	Refunded order, optional for future use

For the current project version, the system mainly uses `SUCCESS`. Failed order attempts are not saved into the `orders` table.

---

4. Table Details

4.1 app_users

Purpose

The `app_users` table stores user login information and role-based access information. The system uses this table for custom authentication instead of Supabase Auth.

Columns

Column	Type	Constraint / Description
id	uuid	Primary key, default `gen_random_uuid()`
username	citext	Unique, not null, used for login
email	citext	Optional email
password_hash	text	Not null, stores hashed password
full_name	text	Not null, display name
role	user_role	Not null, ADMIN or STAFF
status	user_status	Not null, default ACTIVE
last_login_at	timestamptz	Last successful login time
created_at	timestamptz	Default current timestamp
updated_at	timestamptz	Auto update timestamp
deleted_at	timestamptz	Soft delete timestamp

Main Rules

- Username must be unique.
- Password is stored as hash, not plaintext.
- Only ACTIVE users can login.
- ADMIN can access admin management screens.
- STAFF can access POS and Order History.

Sample Accounts

Username	Password	Role	Status
admin	Admin123@	ADMIN	ACTIVE
staff	Staff123@	STAFF	ACTIVE

Username	Password	Role	Status
staff2	Staff123@	STAFF	ACTIVE
staff_inactive	Staff123@	STAFF	INACTIVE

4.2 products

Purpose

The `products` table stores menu items that can be sold at the POS screen.

Columns

Column	Type	Constraint / Description
id	uuid	Primary key, default `gen_random_uuid()`
name	text	Not null, product name
price	numeric(12,2)	Not null, must be greater than 0
status	product_status	Not null, default ACTIVE
description	text	Optional product description
image_path	text	Optional product image path
created_by	uuid	Foreign key to `app_users(id)`, nullable
created_at	timestamptz	Default current timestamp
updated_at	timestamptz	Auto update timestamp
deleted_at	timestamptz	Soft delete timestamp

Main Rules

- Product name cannot be empty.
- Product price must be greater than 0.
- Inactive products must not be displayed or selected on POS.
- A product must have an active recipe before it can be sold.
- Product deletion should use soft delete by setting `deleted_at`.

4.3 ingredients

Purpose

The `ingredients` table stores inventory materials used to make products.

Columns

Column	Type	Constraint / Description
id	uuid	Primary key, default `gen_random_uuid()`
name	text	Not null, ingredient name
unit	ingredient_unit	Not null, GRAM / ML / PIECE
current_stock	numeric(12,3)	Not null, default 0, must be >= 0
low_stock_threshold	numeric(12,3)	Not null, default 0, must be >= 0
created_by	uuid	Foreign key to `app_users(id)`, nullable
created_at	timestamptz	Default current timestamp
updated_at	timestamptz	Auto update timestamp
deleted_at	timestamptz	Soft delete timestamp

Main Rules

- Ingredient name cannot be empty.
- Current stock cannot be negative.
- Low-stock threshold cannot be negative.
- The report screen uses `current_stock <= low_stock_threshold` to detect low-stock ingredients.
- Ingredient deletion should use soft delete by setting `deleted_at`.

4.4 recipes

Purpose

The `recipes` table stores recipe headers. Each product has one active recipe.

Columns

Column	Type	Constraint / Description
id	uuid	Primary key, default `gen_random_uuid()`

Column	Type	Constraint / Description
product_id	uuid	Foreign key to `products(id)`, not null
note	text	Optional recipe note
created_by	uuid	Foreign key to `app_users(id)`, nullable
created_at	timestamptz	Default current timestamp
updated_at	timestamptz	Auto update timestamp
deleted_at	timestamptz	Soft delete timestamp

Main Rules

- Each product should have only one active recipe.
- A recipe without recipe items is not valid for POS selling.
- A product without recipe must not be orderable.

4.5 recipe_items

Purpose

The `recipe_items` table stores ingredient lines for each recipe.

Columns

Column	Type	Constraint / Description
id	uuid	Primary key, default `gen_random_uuid()`
recipe_id	uuid	Foreign key to `recipes(id)`, not null
ingredient_id	uuid	Foreign key to `ingredients(id)`, not null
quantity_required	numeric(12,3)	Not null, must be greater than 0
created_at	timestamptz	Default current timestamp

Main Rules

- Quantity required must be greater than 0.
- One ingredient should appear only once in the same recipe.
- During order creation, the system calculates required stock by multiplying `quantity_required` by ordered quantity.

Example

Milk Coffee recipe:

Ingredient	Quantity Required
Coffee bean	20 GRAM
Milk	100 ML
Sugar	10 GRAM

If Staff orders 2 Milk Coffee, the system must deduct:

Ingredient	Deducted Quantity
Coffee bean	40 GRAM
Milk	200 ML
Sugar	20 GRAM

4.6 orders

Purpose

The `orders` table stores successful POS orders.

Columns

Column	Type	Constraint / Description
id	uuid	Primary key, default `gen_random_uuid()`
order_code	text	Unique order code
staff_id	uuid	Foreign key to `app_users(id)`, nullable if user deleted
total_amount	numeric(12,2)	Not null, must be >= 0
status	order_status	Not null, default SUCCESS
note	text	Optional note
created_at	timestamptz	Default current timestamp
updated_at	timestamptz	Auto update timestamp

Main Rules

- Only successful orders are stored.
- Failed order attempts are not stored in this table.
- Reports only calculate successful orders.
- Staff creates orders from the POS screen.
- Order must be created only if all products are valid and all ingredients have enough stock.

4.7 order_items

Purpose

The `order_items` table stores detailed product lines inside each order.

Columns

Column	Type	Constraint / Description
id	uuid	Primary key, default `gen_random_uuid()`
order_id	uuid	Foreign key to `orders(id)`, not null
product_id	uuid	Foreign key to `products(id)`, nullable if product deleted
product_name_snapshot	text	Product name at selling time
quantity	integer	Not null, must be greater than 0
unit_price	numeric(12,2)	Not null, must be greater than 0
subtotal	numeric(12,2)	Not null, must be >= 0
created_at	timestamptz	Default current timestamp

Main Rules

- Quantity must be greater than 0.
- Unit price must be greater than 0.
- Subtotal should equal `quantity * unit_price`.
- Product name and unit price are stored as snapshots so old orders are not affected when product information changes later.

4.8 stock_transactions

Purpose

The `stock_transactions` table records all stock changes, including stock import, stock adjustment, and stock deduction from successful orders.

Columns

Column	Type	Constraint / Description
id	uuid	Primary key, default `gen_random_uuid()`
ingredient_id	uuid	Foreign key to `ingredients(id)`, nullable if ingredient deleted
type	stock_transaction_type	Not null, IMPORT / ORDER_DEDUCT / ADJUST
quantity	numeric(12,3)	Not null, must be greater than 0
before_stock	numeric(12,3)	Not null, must be >= 0
after_stock	numeric(12,3)	Not null, must be >= 0
order_id	uuid	Foreign key to `orders(id)`, nullable
note	text	Optional note
created_by	uuid	Foreign key to `app_users(id)`, nullable
created_at	timestamptz	Default current timestamp

Main Rules

- Quantity must always be positive.
- `before_stock` and `after_stock` must not be negative.
- For IMPORT, `after_stock = before_stock + quantity`.
- For ORDER_DEDUCT, `after_stock = before_stock - quantity`.
- If order creation fails, no ORDER_DEDUCT transaction is created.
- Every successful order that deducts stock should create stock transaction records.

---

5. Views

5.1 v_pos_available_products

Purpose

This view shows products that can be displayed on the POS screen.

Logic

A product is available on POS only when:

- Product is not deleted.
- Product status is ACTIVE.
- Product has an active recipe.
- Recipe has at least one recipe item.

Expected Usage

The Staff POS screen should use this view or equivalent backend query to display only valid sellable products.

5.2 v_low_stock_ingredients

Purpose

This view shows ingredients that are low in stock.

Logic

An ingredient is low stock when:

current_stock <= low_stock_threshold

Expected Usage

The Reports screen uses this view to show low-stock ingredients.

5.3 v_daily_revenue

Purpose

This view summarizes daily revenue and order count.

Logic

The view groups successful orders by order date.

Output Meaning

Field	Meaning
order_date	Date of order
total_orders	Number of successful orders
total_revenue	Sum of successful order total amounts

5.4 v_best_selling_products

Purpose

This view shows product sales ranking.

Logic

The view joins `orders` and `order_items`, then groups data by product and sums quantity.

Output Meaning

Field	Meaning
product_id	Product ID
product_name_snapshot	Product name at selling time
total_quantity	Total sold quantity
total_revenue	Total revenue by product

5.5 v_order_history

Purpose

This view supports the Order History screen.

Logic

The view returns order summary information, including order code, staff name, total amount, status, created date, and item summary if implemented.

Expected Usage

Staff uses Order History to view successful orders and compare order details with submitted data.

6. Relationships

6.1 Relationship Summary

Relationship	Description
app_users 1 - N products	Admin user can create many products
app_users 1 - N ingredients	Admin user can create many ingredients
app_users 1 - N recipes	Admin user can create many recipes
app_users 1 - N orders	Staff user can create many orders
app_users 1 - N stock_transactions	User can create many stock transactions
products 1 - 1 recipes	One product has one active recipe
recipes 1 - N recipe_items	One recipe has many ingredient lines
ingredients 1 - N recipe_items	One ingredient can be used in many recipes
orders 1 - N order_items	One order has many order items
products 1 - N order_items	One product can appear in many order items
orders 1 - N stock_transactions	One order can create many stock deduction transactions
ingredients 1 - N stock_transactions	One ingredient has many stock movement records

6.2 Text ERD

app_users
├── products.created_by
├── ingredients.created_by
├── recipes.created_by
├── orders.staff_id
└── stock_transactions.created_by

products
└── recipes
└── recipe_items
└── ingredients

orders
├── order_items
│ └── products

└──── stock_transactions
└──── ingredients

---

7. Core Business Rules Covered by Database

Rule ID	Business Rule	Database Support
BR01	User must login before using system functions	`app_users` stores login accounts
BR02	Admin can manage product, ingredient, stock, recipe, reports	`app_users.role = ADMIN`
BR03	Staff can create POS order and view order history	`app_users.role = STAFF`
BR04	Ingredient name cannot be empty	`ingredients.name not null` and check trim
BR05	Stock quantity and import quantity cannot be negative	`ingredients.current_stock >= 0`, transaction checks
BR06	Product price must be greater than 0	`products.price > 0`
BR07	Recipe quantity must be greater than 0	`recipe_items.quantity_required > 0`
BR08	Product must have valid recipe before order	`recipes` and `recipe_items` required for POS query
BR09	Order only succeeds if all ingredients have enough stock	Backend checks stock before inserting order
BR10	If order fails, no ingredient is deducted	Backend transaction and no stock transaction on failure
BR11	If order succeeds, stock decreases by recipe x quantity	Backend deducts stock and creates stock transactions
BR12	Reports only count successful orders	Report views filter successful orders

---

8. Indexes and Constraints

Important Constraints

Table	Constraint
app_users	username unique

Table	Constraint
products	price > 0
products	active product name should not duplicate active product name
ingredients	current_stock >= 0
ingredients	low_stock_threshold >= 0
recipes	one active recipe per product
recipe_items	quantity_required > 0
recipe_items	unique recipe_id + ingredient_id
orders	order_code unique
orders	total_amount >= 0
order_items	quantity > 0
order_items	unit_price > 0
order_items	subtotal >= 0
stock_transactions	quantity > 0
stock_transactions	before_stock >= 0
stock_transactions	after_stock >= 0

Important Indexes

Index Target	Purpose
app_users(username)	Fast login lookup
products(status)	Fast product filtering
products(deleted_at)	Soft delete filtering
ingredients(deleted_at)	Soft delete filtering
recipes(product_id)	Fast recipe lookup by product
recipe_items(recipe_id)	Fast recipe item lookup
orders(created_at)	Fast order history and report filtering
order_items(order_id)	Fast order detail lookup
stock_transactions(ingredient_id)	Fast ingredient stock movement lookup
stock_transactions(created_at)	Fast stock transaction history filtering

9. Trigger / Function Usage

The database has only simple update timestamp logic.

Function

`set_updated_at()`

Purpose:

- Automatically update the `updated_at` column when a row is updated.

Triggers

The following triggers may be used:

Trigger	Table	Purpose
trg_app_users_updated_at	app_users	Update `updated_at`
trg_products_updated_at	products	Update `updated_at`
trg_ingredients_updated_at	ingredients	Update `updated_at`
trg_recipes_updated_at	recipes	Update `updated_at`
trg_orders_updated_at	orders	Update `updated_at`

Important Note

The database does not use triggers to deduct stock or create orders. Stock deduction logic is handled in backend service code to make the system easier to test, debug, and explain.

---

10. POS Order Processing Logic

The POS order logic is handled by the backend.

Flow

1. Staff submits cart items from POS screen.
2. Backend checks whether Staff is logged in.
3. Backend checks whether user role is STAFF.
4. Backend rejects empty cart.
5. Backend loads all products in cart.
6. Backend rejects missing product.
7. Backend rejects inactive product.
8. Backend loads recipe for each product.

9. Backend rejects product without recipe.
10. Backend loads recipe items.
11. Backend calculates total order amount.
12. Backend calculates total required ingredients.
13. Backend checks current stock for each ingredient.
14. Backend rejects order if any ingredient is insufficient.
15. Backend starts database transaction.
16. Backend inserts `orders`.
17. Backend inserts `order_items`.
18. Backend updates `ingredients.current_stock`.
19. Backend inserts `stock_transactions` with type ORDER_DEDUCT.
20. Backend commits transaction.
21. Backend returns order code and total amount.

Failure Rule

If any validation fails:

- No order is inserted.
- No order item is inserted.
- No stock is deducted.
- No stock transaction is created.
- Reports remain unchanged.

---

11. Report Logic

Reports are calculated from successful orders and current inventory data.

Daily Revenue

Source:

- `orders`

Condition:

- `status = 'SUCCESS'`

Formula:

- total revenue = sum of `orders.total_amount`
- total orders = count of successful orders

Best-Selling Products

Source:

- `orders`
- `order_items`

Condition:

- Order status is SUCCESS.

Formula:

- total quantity = sum of `order_items.quantity`
- total revenue = sum of `order_items.subtotal`

Low-Stock Ingredients

Source:

- `ingredients`

Condition:

- `current_stock <= low_stock_threshold`
- `deleted_at is null`

---

12. Sample Test Data

Users

Username	Password	Role	Expected Result
admin	Admin123@	ADMIN	Login to Admin Dashboard
staff	Staff123@	STAFF	Login to Staff POS Dashboard
staff_inactive	Staff123@	STAFF	Login should be rejected

Products

Product	Price	Status	Recipe
Milk Coffee	30000	ACTIVE	Has recipe
Black Coffee	25000	ACTIVE	Has recipe

Product	Price	Status	Recipe
Inactive Test Product	20000	INACTIVE	Has recipe but not sellable
No Recipe Product	20000	ACTIVE	No recipe, used to test missing recipe

Ingredients

Ingredient	Unit	Example Stock	Threshold
Coffee bean	GRAM	1000	100
Milk	ML	1000	100
Sugar	GRAM	50	100

Recipes

Milk Coffee:

Ingredient	Quantity
Coffee bean	20 GRAM
Milk	100 ML
Sugar	10 GRAM

Black Coffee:

Ingredient	Quantity
Coffee bean	20 GRAM
Sugar	5 GRAM

---

13. Database Design Notes

1. The database uses soft delete for products, ingredients, recipes, and users.
2. Failed orders are not stored because the system focuses on successful POS transaction records.
3. Order item stores product name and price snapshot to preserve historical order accuracy.
4. Reports are based only on successful orders.
5. Stock transaction records are used to trace all inventory changes.
6. Stock deduction is not hidden inside database triggers. It is implemented in backend transaction logic for easier manual testing and debugging.
7. Supabase is used only as PostgreSQL cloud database. Frontend should not directly access database secrets.

8. Backend is responsible for authentication, authorization, validation, order transaction, and report API.